

Rust逆向之类

class在Rust中不再是关键字，要定义类只能通过struct关键字来定义结构体。

1.成员函数

下面是一个具有new和add两个成员函数的类的例子：

```
struct Test {  
    x: i32,  
    y: i32,  
}  
  
impl Test {  
    fn new(x: i32, y: i32) -> Test {  
        Self {  
            x,  
            y,  
        }  
    }  
    fn add(&self, z: i32) -> i32 {  
        self.x + self.y + self.z  
    }  
}  
  
fn main() {  
    let t = Test::new(3, 4);  
    let x = t.add(5);  
}
```

从下面的汇编代码可以看出，当调用的成员函数的参数没有self关的时候，这个成员函数和普通函数传参方式没有区别。当成员函数的参数有self的时候，函数的第一个参数就会是类变量的地址：

```

.text:000000001400011F0 ; void __fastcall test_pro::main()
.text:000000001400011F0 test_pro__main proc near
.text:000000001400011F0
.text:000000001400011F0             sub     rsp, 38h
.text:000000001400011F4             mov     ecx, 3             ; int
.text:000000001400011F9             mov     edx, 4             ; int
.text:000000001400011FE             call    test_pro__Test__new
.text:00000000140001203             mov     [rsp+30h], edx    ; 给变量t赋值
.text:00000000140001207             mov     [rsp+2Ch], eax
.text:0000000014000120B             lea     rcx, [rsp+2Ch]    ; 将变量t的地址赋给rcx
.text:00000000140001210             mov     edx, 5             ; 传递参数z
.text:00000000140001215             call    test_pro__Test__add
.text:0000000014000121A             mov     [rsp+34h], eax    ; 给变量x赋值
.text:0000000014000121E             add     rsp, 38h
.text:00000000140001222             retn
.text:00000000140001222 test_pro__main end

```

new函数的内部会保存一次参数t，在将t赋值到一块内存中，在从这块内存中将值取出作为返回值：

```

.text:00000000140001150 ; test_pro::Test __fastcall test_pro::Test::new(int, int)
.text:00000000140001150 test_pro__Test__new proc near
.text:00000000140001150
.text:00000000140001150             sub     rsp, 10h
.text:00000000140001154             mov     [rsp+8], ecx
.text:00000000140001158             mov     [rsp+0Ch], edx
.text:0000000014000115C             mov     [rsp], ecx
.text:0000000014000115F             mov     [rsp+4], edx
.text:00000000140001163             mov     eax, [rsp]
.text:00000000140001166             mov     edx, [rsp+4]
.text:0000000014000116A             add     rsp, 10h
.text:0000000014000116E             retn
.text:0000000014000116E test_pro__Test__new endp

```

而在add函数内部，会通过rcx中保存的变量t的地址，来获取t.x和t.y的值。然后将t.x，t.y和参数z相加的值作为返回值：

```

.text:0000000140001170 ; int __fastcall test_pro::Test::add(int)
.text:0000000140001170 test_pro__Test__add proc near
.text:0000000140001170
.text:0000000140001170             sub     rsp, 48h
.text:0000000140001174             mov     [rsp+30h], edx ; 保存参数z的值
.text:0000000140001178             mov     [rsp+38h], rcx ; 保存t的地址
.text:000000014000117D             mov     [rsp+44h], edx
.text:0000000140001181             mov     eax, [rcx] ; eax=t.x
.text:0000000140001183             add     eax, [rcx+4] ; eax=t.x+t.y
.text:0000000140001186             mov     [rsp+34h], eax ; 保存t.x+t.y
.text:000000014000118A             seto    al
.text:000000014000118D             test   al, 1
.text:000000014000118F             jnz     short loc_1400011A8 ; 整数溢出则跳转
.text:0000000140001191             mov     ecx, [rsp+30h] ; ecx=z
.text:0000000140001195             mov     eax, [rsp+34h] ; eax=t.x+t.y
.text:0000000140001199             add     eax, ecx ; eax=t.x+t.y+z
.text:000000014000119B             mov     [rsp+2Ch], eax ; 保存t.x+t.y+z
.text:000000014000119F             seto    al
.text:00000001400011A2             test   al, 1
.text:00000001400011A4             jnz     short loc_1400011C9 ; 整型溢出则跳转
.text:00000001400011A6             jmp     short loc_1400011C0 ; 将t.x+t.y+z的值作
为返回值
.text:00000001400011A8 ; -----
-----
.text:00000001400011A8
.text:00000001400011A8 loc_1400011A8: ; CODE XREF:
test_pro__Test__add+1F↑j
.text:00000001400011A8             lea     rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...
.text:00000001400011AF             lea     r8, off_140019360 ; "src\\main.rs"
.text:00000001400011B6             mov     edx, 1Ch
.text:00000001400011BB             call    _ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
.text:00000001400011C0 ; -----
-----
.text:00000001400011C0
.text:00000001400011C0 loc_1400011C0: ; CODE XREF:
test_pro__Test__add+36↑j
.text:00000001400011C0             mov     eax, [rsp+2Ch] ; 将t.x+t.y+z的值作为返
返回值
.text:00000001400011C4             add     rsp, 48h
.text:00000001400011C8             retn
.text:00000001400011C9 ; -----
-----
.text:00000001400011C9
.text:00000001400011C9 loc_1400011C9: ; CODE XREF:
test_pro__Test__add+34↑j
.text:00000001400011C9             lea     rcx, aAttemptToAddWi ; "attempt to add
with overflowcalled `Opt"...

```

```

.text:000000001400011D0      lea      r8, off_140019360 ; "src\\main.rs"
.text:000000001400011D7      mov      edx, 1Ch
.text:000000001400011DC      call
_ZN4core9panicking5panic17h61d0277f5e1a7407E ;
core::panicking::panic::h61d0277f5e1a7407
.text:000000001400011DC ; -----
-----
.text:000000001400011E1      db  0CCh
.text:000000001400011E1 test_pro__Test__add endp

```

所以在成员函数这一块，Rust和C/C++是一样的。成员函数的参数中没有self的时候，对该函数的调用和普通函数在内存中是一样的。而如果成员函数参数有self，则会将类变量的地址作为第一个参数传入。

2.组合

Rust中的类是不支持继承的，想要实现继承的功能，只能像下面的例子一样，通过组合来实现：

```

struct Test {
    x: i32,
    y: i32,
}

impl Test {
    fn new(x: i32, y: i32) -> Test {
        Self {
            x,
            y,
        }
    }
    fn add(&self, z: i32) -> i32 {
        self.x + self.y + z
    }
}

struct Test1 {
    t: Test,
    c: char,
}

impl Test1 {
    fn add(&self) -> i32 {
        self.t.x + self.t.y + self.c as i32
    }
}

fn main() {
    let t = Test::new(3, 4);
    let t1 = Test1 {
        t: t,
        c: 'a',
    };

    let x = t1.t.add(5);
    let y = t1.add();
}

```

从下面的反汇编可以知道，t中的值会被赋值到t1的内存中。通过t1.t.add(5)来调用t中add函数的时候，会先获取t1的地址，然后再计算得到t1中保存t的地址，再将其作为第一个参数传入。

```

.text:0000000140001270 ; void __fastcall test_pro::main()
.text:0000000140001270 test_pro__main proc near
.text:0000000140001270
.text:0000000140001270         sub     rsp, 48h
.text:0000000140001274         mov     ecx, 3             ; int
.text:0000000140001279         mov     edx, 4             ; int
.text:000000014000127E         call    test_pro__Test__new
.text:0000000140001283         mov     [rsp+38h], eax     ; 用返回值给变量t赋值
.text:0000000140001287         mov     [rsp+3Ch], edx
.text:000000014000128B         mov     [rsp+30h], eax     ; t1.x=t.x
.text:000000014000128F         mov     [rsp+34h], edx     ; t1.y=t.y
.text:0000000140001293         mov     dword ptr [rsp+2Ch], 'a' ; t1.c='a'
.text:000000014000129B         lea     rcx, [rsp+2Ch]     ; rcx=t1的地址
.text:00000001400012A0         add     rcx, 4             ; rcx=t1中成员t的地址
.text:00000001400012A4         mov     edx, 5
.text:00000001400012A9         call    test_pro__Test__add
.text:00000001400012AE         mov     [rsp+40h], eax     ; 给x变量赋值
.text:00000001400012B2         lea     rcx, [rsp+2Ch]     ; rcx=t1的地址
.text:00000001400012B7         call    test_pro__Test1__add
.text:00000001400012BC         mov     [rsp+44h], eax     ; 给变量y赋值
.text:00000001400012C0         add     rsp, 48h
.text:00000001400012C4         retn
.text:00000001400012C4 test_pro__main endp

```

上面两次调用的add函数，在不同的内存地址中，也就是两个不同的函数。通过获取到相应类变量的地址空间，将其作为第一个参数来分别调用不同的add函数，就可以正确执行两个不同add函数中的代码。两个add函数的代码没什么特别的，就不放出来了。

3.特性

特性(trait)类似于接口，用来标识不同的类拥有相同的方法。trait和接口不同的是，trait可以定义默认方法。如果不定义默认方法，为类实现trait的时候，就必须实现trait中定义的函数。以下是trait的一个基本用法：

```

trait PrintTrait {
    fn print_value(&self) {
        println!("Default print");
    }
}

struct Test {
    x: i32,
    y: i32,
}

impl Test {
    fn new(x: i32, y: i32) -> Test {
        Self {
            x,
            y,
        }
    }
    fn add(&self, z: i32) -> i32 {
        self.x + self.y + z
    }
}

impl PrintTrait for Test {
    fn print_value(&self) {
        println!("x={},y={}", self.x, self.y);
    }
}

struct Test1 {
    t: Test,
    c: char,
}

impl Test1 {
    fn add(&self) -> i32 {
        self.t.x + self.t.y + self.c as i32
    }
}

impl PrintTrait for Test1 {
    fn print_value(&self) {
        println!("x={},y={},c={}", self.t.x, self.t.y, self.c);
    }
}

struct Test2 {}
impl PrintTrait for Test2 {

```

```

}

fn main() {
    let t = Test::new(3, 4);
    let t1 = Test1 {
        t: Test::new(5, 6),
        c: 'a',
    };
    let t2 = Test2 {};

    t.print_value();
    t1.print_value();
    t2.print_value();
}

```

从下面的反汇编可以看到，三个不同的类会调用不同的函数，前两个调用的是为这两个类编写的print_value函数，最后一个则是默认的print_value函数。

```

.text:0000000140001590 ; void __fastcall test_pro::main()
.text:0000000140001590 test_pro__main proc near
.text:0000000140001590
.text:0000000140001590         sub     rsp, 48h
.text:0000000140001594         mov     ecx, 3          ; int
.text:0000000140001599         mov     edx, 4          ; int
.text:000000014000159E         call    test_pro__Test__new
.text:00000001400015A3         mov     [rsp+34h], edx    ; 初始化t
.text:00000001400015A7         mov     [rsp+30h], eax
.text:00000001400015AB         mov     ecx, 5          ; int
.text:00000001400015B0         mov     edx, 6          ; int
.text:00000001400015B5         call    test_pro__Test__new
.text:00000001400015BA         mov     [rsp+3Ch], eax    ; 初始化t1
.text:00000001400015BE         mov     [rsp+40h], edx
.text:00000001400015C2         mov     dword ptr [rsp+38h], 'a'
.text:00000001400015CF         lea     rcx, [rsp+30h]    ; rcx=变量t地址
.text:00000001400015D4         call    test_pro__impl$1__print_value
.text:00000001400015D9         lea     rcx, [rsp+38h]    ; rcx=变量t1地址
.text:00000001400015DE         call    test_pro__impl$3__print_value
.text:00000001400015E3         lea     rcx, [rsp+47h]    ; rcx=变量t2地址
.text:00000001400015E3         call    _ZN8test_pro10PrintTrait11print_value17he043b5a615cc11a5E ;
test_pro::PrintTrait::print_value::he043b5a615cc11a5
.text:00000001400015E8         nop
.text:00000001400015E9         add     rsp, 48h
.text:00000001400015ED         retn
.text:00000001400015ED test_pro__main endp

```

因此，可以得出结论。为类实现trait，其实就是将该trait中指定的函数，增加到这个类的成员函数中。而编写trait的那些限制，比如为某个类实现某个trait，需要实现该trait中未默认实现的函数，

都是编译器层面的检查。

以下Rust代码，是将trait可以作为参数和返回值的例子：

```
trait PrintTrait {
    fn print_value(&self) {
        println!("Default print");
    }
}

struct Test {
    x: i32,
    y: i32,
}

impl Test {
    fn new(x: i32, y: i32) -> Test {
        Self {
            x,
            y,
        }
    }
    fn add(&self, z: i32) -> i32 {
        self.x + self.y + z
    }
}

impl PrintTrait for Test {
    fn print_value(&self) {
        println!("x={},y={}", self.x, self.y);
    }
}

fn test_return_trait(t: impl PrintTrait) -> impl PrintTrait {
    Test {
        x: 1,
        y: 2,
    }
}

fn main() {
    let t = Test {
        x: 5,
        y: 6,
        c: 'a',
    };

    let t1 = test_return_trait(t);
}
```

从反汇编代码可以看到，函数调用的时候就是把变量t的地址作为参数传入。函数返回以后，在根据返回值赋值变量t1：

```
.text:0000000140001190 ; void __fastcall test_pro::main()
.text:0000000140001190 test_pro__main proc near
.text:0000000140001190
.text:0000000140001190             sub     rsp, 38h
.text:0000000140001194             mov     ecx, 5             ; int
.text:0000000140001199             mov     edx, 6             ; int
.text:000000014000119E             call    test_pro__Test__new
.text:00000001400011A3             mov     [rsp+28h], eax     ; 初始化t1
.text:00000001400011A7             mov     [rsp+2Ch], edx
.text:00000001400011AB             mov     dword ptr [rsp+24h], 'a'
.text:00000001400011B3             lea     rcx, [rsp+24h]     ; rcx等于t变量地址
.text:00000001400011B8             call    _ZN8test_pro17test_return_trait17h9054289424d9f894E ;
_ZN8test_pro17test_return_trait17h9054289424d9f894
test_pro::test_return_trait::h9054289424d9f894
.text:00000001400011BD             mov     [rsp+30h], eax     ; 为变量t1赋值
.text:00000001400011C1             mov     [rsp+34h], edx
.text:00000001400011C5             add     rsp, 38h
.text:00000001400011C9             retn
.text:00000001400011C9 test_pro__main endp
```

test_return_trait的汇编代码，也没什么特别的。所以，所谓的将trait作为参数和返回值，也是编译器层面的操作。只是在调用的时候，会检查该类是否实现了这个trait，实现了就将这个类进行参数或者返回值传递。

```
.text:0000000140001150 ; test_pro::Test __fastcall
test_pro::test_return_trait::h9054289424d9f894(test_pro::Test1)
.text:0000000140001150 _ZN8test_pro17test_return_trait17h9054289424d9f894E proc near
.text:0000000140001150
.text:0000000140001150             push    rax
.text:0000000140001151             mov     dword ptr [rsp], 1
.text:0000000140001158             mov     dword ptr [rsp+4], 2
.text:0000000140001160             mov     eax, [rsp]         ; eax=Test.t
.text:0000000140001163             mov     edx, [rsp+4]       ; edx=Test.y
.text:0000000140001167             pop     rcx
.text:0000000140001168             retn
.text:0000000140001168 _ZN8test_pro17test_return_trait17h9054289424d9f894E endp
```

4.多态

因为Rust没有继承，所以Rust就不能像C/C++那样实现多态。Rust需要借助trait才能实现多态，下面是一个简单的例子：

```

trait PrintTrait {
    fn print_value(&self) {
        println!("Default print");
    }
}

struct Test {
    x: i32,
    y: i32,
}

impl Test {
    fn new(x: i32, y: i32) -> Test {
        Self {
            x,
            y,
        }
    }
    fn add(&self, z: i32) -> i32 {
        self.x + self.y + z
    }
}

impl PrintTrait for Test {
    fn print_value(&self) {
        println!("x={},y={}", self.x, self.y);
    }
}

struct Test1 {
    t: Test,
    c: char,
}

impl Test1 {
    fn add(&self) -> i32 {
        self.t.x + self.t.y + self.c as i32
    }
}

impl PrintTrait for Test1 {
    fn print_value(&self) {
        println!("x={},y={},c={}", self.t.x, self.t.y, self.c);
    }
}

struct Test2 {}
impl PrintTrait for Test2 {

```

```

}

fn test_print_trait(t: &dyn PrintTrait) {
    t.print_value();
}

fn main() {

    let t = Test::new(3, 4);
    let t1 = Test1 {
        t: Test::new(5, 6),
        c: 'a',
    };

    let t2 = Test2 {};

    test_print_trait(&t);
    test_print_trait(&t1);
    test_print_trait(&t2);
}

```

从如下反汇编可以看到，在调用test_print_trait的时候，除了将变量地址作为第一个参数，还会分别将三个不同的地址作为第二个参数传递：

```

.text:00000001400015E0 ; void __fastcall test_pro::main()
.text:00000001400015E0 test_pro__main proc near
.text:00000001400015E0
.text:00000001400015E0             sub     rsp, 48h
.text:00000001400015E4             mov     ecx, 3             ; int
.text:00000001400015E9             mov     edx, 4             ; int
.text:00000001400015EE             call    test_pro__Test__new
.text:00000001400015F3             mov     [rsp+34h], edx     ; 初始化变量t
.text:00000001400015F7             mov     [rsp+30h], eax
.text:00000001400015FB             mov     ecx, 5             ; int
.text:0000000140001600             mov     edx, 6             ; int
.text:0000000140001605             call    test_pro__Test__new
.text:000000014000160A             mov     [rsp+3Ch], eax     ; 初始化变量t1
.text:000000014000160E             mov     [rsp+40h], edx
.text:0000000140001612             mov     dword ptr [rsp+38h], 'a'
.text:000000014000161A             lea     rcx, [rsp+30h]     ; rcx=变量t地址
.text:000000014000161F             lea     rdx,
impl$_test_pro__Test__test_pro__PrintTrait__vtable$
.text:0000000140001626             call    test_pro__test_print_trait
.text:000000014000162B             lea     rcx, [rsp+38h]     ; rcx=变量t1地址
.text:0000000140001630             lea     rdx,
impl$_test_pro__Test1__test_pro__PrintTrait__vtable$
.text:0000000140001637             call    test_pro__test_print_trait
.text:000000014000163C             lea     rcx, [rsp+47h]     ; rcx=变量t2地址
.text:0000000140001641             lea     rdx,
impl$_test_pro__Test2__test_pro__PrintTrait__vtable$
.text:0000000140001648             call    test_pro__test_print_trait
.text:000000014000164D             nop
.text:000000014000164E             add     rsp, 48h
.text:0000000140001652             retn
.text:0000000140001652 test_pro__main endp

```

而这三个地址，分别是三个具有4个元素的i64类型的数组。数组中的最后一个元素，则是这些类对应的print_trait函数的地址：

```

.rdata:000000014001B4A0 ; impl$<test_pro::Test, test_pro::PrintTrait>::vtable_type$
impl__test_pro::Test__test_pro::PrintTrait__vtable_
.rdata:000000014001B4A0 impl$__test_pro__Test__test_pro__PrintTrait__vtable$
impl$<test_pro::Test, test_pro::PrintTrait>::vtable_type$ <offset
_ZN4core3ptr35drop_in_place$LT$test_pro__Test$GT$17h64d26293a64b69c7E,\
.rdata:000000014001B4A0 ; DATA XREF:
test_pro__main+3F↑o
.rdata:000000014001B4A0
8, 4, \ ; core::ptr::drop_in_place$LT$test_pro..Test$GT$::h64d26293a64b69c7
.rdata:000000014001B4A0
offset test_pro__impl$1__print_value>
.rdata:000000014001B4C0 ; impl$<test_pro::Test1, test_pro::PrintTrait>::vtable_type$
impl__test_pro::Test1__test_pro::PrintTrait__vtable_
.rdata:000000014001B4C0 impl$__test_pro__Test1__test_pro__PrintTrait__vtable$
impl$<test_pro::Test1, test_pro::PrintTrait>::vtable_type$ <offset
_ZN4core3ptr36drop_in_place$LT$test_pro__Test1$GT$17h81423fddbbaa2795aE,\
.rdata:000000014001B4C0 ; DATA XREF:
test_pro__main+50↑o
.rdata:000000014001B4C0
0Ch, 4, \ ; core::ptr::drop_in_place$LT$test_pro..Test1$GT$::h81423fddbbaa2795a
.rdata:000000014001B4C0
offset test_pro__impl$3__print_value>
.rdata:000000014001B4E0 ; impl$<test_pro::Test2, test_pro::PrintTrait>::vtable_type$
impl__test_pro::Test2__test_pro::PrintTrait__vtable_
.rdata:000000014001B4E0 impl$__test_pro__Test2__test_pro__PrintTrait__vtable$
impl$<test_pro::Test2, test_pro::PrintTrait>::vtable_type$ <offset
_ZN4core3ptr36drop_in_place$LT$test_pro__Test2$GT$17h8fc5ee0d70fa2198E,\
.rdata:000000014001B4E0 ; DATA XREF:
test_pro__main+61↑o
.rdata:000000014001B4E0
0, 1, \ ; core::ptr::drop_in_place$LT$test_pro..Test2$GT$::h8fc5ee0d70fa2198
.rdata:000000014001B4E0
offset _ZN8test_pro10PrintTrait11print_value17he043b5a615cc11a5E

```

而在print_trait函数中，则会对edx中保存的数组地址偏移0x18。也就是最后一个元素中保存的地址进行调用，这样就调用到了相应的print_trait函数。

```
.text:000000001400015C0 ; void __fastcall
test_pro::test_print_trait(ref$<dyn$<test_pro::PrintTrait> >)
.text:000000001400015C0 test_pro__test_print_trait proc near
.text:000000001400015C0
.text:000000001400015C0             sub     rsp, 38h
.text:000000001400015C4             mov     [rsp+28h], rcx ; 保存rcx
.text:000000001400015C9             mov     [rsp+30h], rdx ; 保存rdx
.text:000000001400015CE             call    qword ptr [rdx+18h] ; 调用print_trait
.text:000000001400015D1             nop
.text:000000001400015D2             add     rsp, 38h
.text:000000001400015D6             retn
.text:000000001400015D6 test_pro__test_print_trait endp
```